

ADR-003 — Mecanismo de comunicación entre el chatbot y el panel

ID	Título	Estado	Fecha
ADR-003	Mecanismo de comunicación entre el chatbot y el panel	Aceptada	27/05/2026

1. Contexto

Como consecuencia de ADR-001, el sistema queda dividido en dos servicios (`my-app` y `chatbot`) que igual deben colaborar: cuando un cliente escribe por WhatsApp, el bot necesita validar que está registrado, listarle sus pólizas, descargar su tarjeta de circulación o crear un siniestro con documentos en el sistema principal. Las rutas concretas ya están prototipadas en `propuesta-de-avance/Usos-routes.md` (validación por teléfono, listar pólizas, solicitudes nuevas, tarjeta, comprobantes, siniestros).

Motivadores que obligan a tomar una decisión sobre el mecanismo de comunicación:

- **Motivador 1 — Patrón de interacción request/response del bot:** cada mensaje entrante del cliente exige una respuesta del bot en segundos (la conversación de WhatsApp es síncrona desde la perspectiva del usuario). El bot necesita el dato (¿este cliente existe?, ¿qué pólizas tiene?) antes de poder responder; no puede contestar y "ya te aviso".
- **Motivador 2 — Volumen acotado por la cadencia humana:** el chatbot atiende a clientes finales escribiendo a mano; el volumen pico realista está muy por debajo del que justificaría una capa de mensajería. No hay procesos batch ni fan-out masivo entre los servicios.
- **Motivador 3 — Seguridad (RNF Confidencialidad e Integridad, Entrega 1):** la API expuesta por `my-app` maneja datos sensibles (DNIs, pólizas, comprobantes de pago). Solo el chatbot —no usuarios anónimos— debe poder consumirla. Se necesita autenticación server-to-server simple y verificable.
- **Motivador 4 — Capacidad del equipo y plazo del PoC:** un equipo de 6 integrantes en un trabajo académico no puede operar broker de mensajes, esquemas Avro/Protobuf, dead-letter queues ni monitoreo de colas además de los dos servicios y la integración WhatsApp.

2. Alternativas consideradas

Alternativa	Ventaja principal en este contexto	Desventaja / Motivo de descarte
Cola de mensajes (RabbitMQ / SQS / Redis Streams)	Desacopla temporalmente bot y panel; si <code>my-app</code> cae, los mensajes se acumulan y se procesan al volver. Buena para picos de carga.	No encaja con el motivador 1: el bot necesita respuesta sincrónica para contestar al cliente (no puede esperar a que la cola procese y vuelva). Suma infraestructura (broker, workers, observabilidad) sin justificación de volumen (motivador 4).
Eventos / webhooks bidireccionales (pub-sub)	Cada servicio publica eventos de dominio; el otro reacciona. Útil para arquitecturas event-driven con muchos consumidores.	Sobrediseño para dos servicios con un patrón petición/respuesta puro. Complica el debug (la causa-efecto deja de ser lineal) y exige idempotencia y deduplicación que no son problemas reales acá.

REST síncrono server-to-server con API Key (elegida)	Petición/respuesta directa, fácil de razonar, fácil de testear con curl y mocks. La autenticación por header X-API-Key da control de acceso simple y suficiente para tráfico server-to-server.	— (esta fue la elegida)
---	--	-------------------------

3. Decisión tomada

Se decide: la comunicación entre el chatbot y el panel será **REST síncrono server-to-server sobre HTTPS**, con autenticación por **header X-API-Key** verificado en cada request de `my-app`. Los endpoints son los enumerados en `propuesta-de-avance/Usos-routes.md` (`GET /api/clients/by-phone/[phone]`, `GET /api/policies?phone=`, `POST /api/policy-requests`, `POST /api/circulation-card`, `POST /api/payment-receipts`, `POST /api/claims`). En el chatbot, la llamada está encapsulada en `chatbot/app/main_system_client.py`, con modo `mock` para desarrollo.

Fundamentación:

- Resuelve el motivador 1 (interacción síncrona):** el bot llama, recibe el dato y responde al cliente en el mismo turno. No hay que orquestar callbacks ni mantener estado intermedio entre llamada y respuesta.
- Resuelve los motivadores 2 y 4 (volumen y capacidad del equipo):** REST sobre HTTPS es el camino más simple y más debugueable; cualquier integrante del grupo lo entiende sin onboarding adicional. No agrega componentes nuevos a operar (broker, workers).
- Resuelve el motivador 3 (seguridad server-to-server):** el header `X-API-Key` es simple de implementar (validar en un middleware Next.js), permite rotar el secreto sin tocar usuarios finales y es suficiente para autenticar a un solo cliente conocido (el chatbot). El patrón ya está en uso en `chatbot/app/main_system_client.py`.

4. Consecuencias

Consecuencias positivas	Trade-offs / costos
El contrato es explícito y versionable (los endpoints están enumerados en <code>Usos-routes.md</code>).	Acoplamiento temporal: si <code>my-app</code> cae o tiene latencia alta, el bot no puede responder. El cliente percibe el caído como un caído del bot, no del panel.
Debug y testing simples: curl, Postman, mocks por archivo (<code>mock_system.py</code> ya existe en el chatbot).	Sin retry / idempotencia automática: una pérdida de paquete en <code>POST /api/claims</code> puede crear el siniestro y perder la confirmación, o no crearlo y dejar al cliente sin respuesta. Hay que tratarlo en cada handler.
API Key permite cortar acceso del bot inmediatamente si se compromete (rotar el secreto).	Sin event log nativo: no queda historial automático de "qué le pidió el bot al panel". Si se necesita auditar interacciones, hay que loggear explícitamente en ambos lados.
No agrega infraestructura nueva (sin broker, sin colas, sin schema registry).	El bot debe conocer la URL pública del panel (variable de entorno <code>MAIN_SYSTEM_BASE_URL</code>). En despliegues con preview environments hay que poblar esa variable por entorno.

Decisiones relacionadas:

- ADR-001 (separación de servicios): este ADR resuelve cómo se hablan los dos servicios separados.
- ADR-004 (despliegue heterogéneo): el panel debe exponer URL pública estable para que el chatbot pueda apuntarle.
- ADR-005 (WhatsApp Cloud API): el patrón síncrono encaja con el flujo entrante de WhatsApp, también request/response.